

## 类的层次结构

- **TickManager** (行情管理器) : 这是顶层类, 有两个子类:
  - **YahooFinanceFileTickManager**: 可能是用于从 Yahoo Finance 文件获取行情数据的管理器。
  - **RandomTickManager**: 可能是用于生成随机行情数据的管理器。
- **CurrencyConversion** (货币转换) : 这个类有三个方法:
  - **removeCommasFromCurrency**: 从货币数据中移除逗号。
  - **readwithPrecision**: 高精度读取。
- **Tick (行情)** : 这是一个单独的类, 可能用于表示行情数据。
- **VirtualBank** (虚拟银行) : 这是一个中间层类, 有两个子类:
  - **Strategy** (策略)  
: 这个类有两个子类:
    - **SmaStrategy**: 可能是简单移动平均策略。
    - **RandomBuySellStrategy**: 可能是随机买卖策略。
  - **TradeManager** (交易管理器) : 这个类有一个子类:
    - **PaperTradeManager**: 可能是模拟交易管理器。

### 1. 数据输入管理 (TickManager 和其子类)

- **TickManager** 是一个接口或抽象类, 负责管理股票交易数据 (通常是每分钟或每天的价格和交易量等信息)。
- **RandomTickManager** 和 **YahooFinanceFileTickManager** 都是 **TickManager** 的实现。它们提供不同的方式来获取 Tick 数据:
  - **RandomTickManager** 使用随机生成的数据模拟市场价格。
  - **YahooFinanceFileTickManager** 从文件中加载历史数据 (如 Yahoo Finance 提供的 CSV 文件)。

### 2. 交易策略 (Strategy 和其实现)

- **Strategy** 是一个抽象基类, 定义了 **processTick** 方法, 所有的具体策略都需要实现这个方法来处理市场数据 (Tick)。
- **RandomBuySellStrategy** 和 **SmaStrategy** 都是 **Strategy** 的具体实现:
  - **RandomBuySellStrategy** 随机生成买入、卖出或保持的信号 (随机策略)。
  - **SmaStrategy** 使用简单移动平均 (SMA) 策略来决定买入或卖出信号。

### 3. 交易管理 (TradeManager 和其实现)

- **TradeManager** 是一个抽象基类, 负责执行实际的买卖操作。
- **PaperTradeManager** 是 **TradeManager** 的实现, 模拟实际交易过程, 包括检查资金是否足够、是否持有股票等。

### 4. 虚拟银行 (VirtualBank)

- `VirtualBank` 用来管理虚拟账户中的余额。每当执行一次买入或卖出操作时，虚拟银行会根据操作调整账户余额。

## 5. 交易 (Trade)

- `Trade` 类表示一笔交易，包括交易的类型（买入、卖出、保持）、数量、价格等。

## 6. 其他辅助类

- `CurrencyConversions` 提供货币格式的转换功能（比如去除逗号）。
- `Tick` 类代表市场的单一数据点（例如，某个时刻的开盘价、收盘价、最高价、最低价等）。

### 如何组合在一起：

1. `YahooFinanceFileTickManager` 读取外部 CSV 文件，将每条数据（`Tick`）按顺序存储，并提供 `getNextTick` 方法供策略使用。
2. `Strategy` 使用市场数据（`Tick`）生成交易信号（买入、卖出或保持）。例如，`SmaStrategy` 根据过去一段时间的平均价格来决定是否买入或卖出股票。
3. `TradeManager` 根据策略生成的信号执行相应的交易。交易管理器会检查是否可以执行交易（例如，是否有足够的资金进行买入，或是否有持有的股票可以卖出）。
4. `VirtualBank` 管理账户余额。每次交易时，银行账户会根据交易情况（如买入时扣款，卖出时增加余额）进行调整。

代码源：

<https://github.com/flouthoc/aku/>.

yang ye:

<https://replit.com/@YeKunlun/akufork?v=1>

一个简单交易模拟系统的核心逻辑循环部分，主要涉及行情数据获取、交易策略应用以及交易执行等操作，整体围绕着一个事件循环来不断处理新的行情数据并进行相应交易操作。

### 行情管理器初始化：

```
TickManager * tick_manager;  
tick_manager = new YourTickManager(input_file);
```

首先声明了一个指向 `TickManager` 类型的指针 `tick_manager`，接着通过 `new` 操作符创建了一个 `YourTickManager` 类的实例，并传入参数 `input_file`（推测是包含行情数据的文件路径之类的参数），用于初始化行情管理器，该管理器后续会负责提供行情数据。

### 交易策略初始化：

```
Strategy * strategy;  
strategy = new YourCustomStrategy();
```

声明了一个指向 `Strategy` 类型的指针 `strategy`，然后同样使用 `new` 操作符创建了 `YourCustomStrategy` 类的实例，用于初始化具体的交易策略，不同的策略类应该实现了不同的交易逻辑，这里初始化的是自定义的策略。

### 交易管理器初始化：

```
TradeManager * trade_manager;
trade_manager = new YourTradeManager(virtual_bank);
```

声明了指向 `TradeManager` 类型的指针 `trade_manager`，并通过 `new` 操作符创建 `YourTradeManager` 类的实例，同时传入参数 `virtual_bank`（可能是代表虚拟银行账户相关的对象，用于管理资金等情况）来初始化交易管理器，它将负责执行具体的交易操作。

**核心事件循环部分：**

```
Tick last_traded_tick;
while(tick_manager->hasNextTick()){
    Tick t = tick_manager->getNextTick();
    last_traded_tick = t;
    Trade tr = strategy->processTick(t);
    trade_manager->performTrade(tr);
}
```

**实例**

```
#include <iostream>
#include <vector>

// 定义行情结构体，用于表示某一时刻的交易数据，比如价格等
struct Tick {
    double price;
    // 可以根据需要添加更多行情相关的数据字段，比如时间戳等
};

// 行情管理器类，负责提供行情数据
class TickManager {
private:
    std::vector<Tick> ticks; // 存储行情数据的容器
    int currentIndex;        // 当前已提供的行情数据索引

public:
    TickManager() : currentIndex(0) {}

    // 模拟加载行情数据，这里简单添加几个示例行情数据
    void loadTicks() {
        Tick t1 = {100.0};
        Tick t2 = {105.0};
        Tick t3 = {102.0};
        ticks.push_back(t1);
        ticks.push_back(t2);
        ticks.push_back(t3);
    }

    bool hasNextTick() {
        return currentIndex < ticks.size();
    }

    Tick getNextTick() {
        return ticks[currentIndex++];
    }
}
```

```

};

// 交易策略抽象基类，不同的策略类可以继承它并实现自己的逻辑
class Strategy {
public:
    virtual Trade processTick(const Tick& tick) = 0;
};

// 简单的示例策略类，比如当价格高于某个阈值就买入
class SimpleBuyStrategy : Strategy {
private:
    double buyThreshold;

public:
    SimpleBuyStrategy(double threshold) : buyThreshold(threshold) {}

    Trade processTick(const Tick& tick) override {
        Trade trade;
        if (tick.price > buyThreshold) {
            trade.isBuy = true;
            trade.quantity = 1; // 简单设定买入数量为1，可调整
            trade.price = tick.price;
        }
        return trade;
    }
};

// 交易结构体，记录交易相关信息
struct Trade {
    bool isBuy;
    int quantity;
    double price;
};

// 交易管理器类，负责执行交易操作
class TradeManager {
public:
    void performTrade(const Trade& trade) {
        if (trade.isBuy) {
            std::cout << "执行买入操作，数量：" << trade.quantity << "，价格：" <<
trade.price << std::endl;
        } else {
            std::cout << "执行卖出操作，数量：" << trade.quantity << "，价格：" <<
trade.price << std::endl;
        }
    }
};

int main() {
    TickManager tickManager;
    tickManager.loadTicks();

    SimpleBuyStrategy strategy(101.0);
    TradeManager tradeManager;

    Tick lastTradedTick;

```

```

    while (tickManager.hasNextTick()) {
        Tick t = tickManager.getNextTick();
        lastTradedTick = t;
        Trade tr = strategy.processTick(t);
        tradeManager.performTrade(tr);
    }

    return 0;
}

```

## 改错：

## In "main.cpp"

### 原代码分析

```

#include <fstream>
#include <iostream>
#include <random>

#include "../include/staticmsgs.h"

#include "../include/Tick.h"
#include "../include/TickManager.h"

#include "../include/RandomTickManager.h"
#include "../include/YahooFinanceFileTickManager.h"

#include "../include/RandomBuySellStrategy.h"
#include "../include/SmaStrategy.h"
#include "../include/Strategy.h"

#include "../include/PaperTradeManager.h"
#include "../include/TradeManager.h"

#include "../include/VirtualBank.h"

using namespace std;

int main(int argc, char **argv) {
    /* 与命令行参数相关的部分 */
    string input_file{"sample_inputs/MARUTI.NS.csv"}; // 输入文件路径（股票价格数据）
    string output_file{"output.csv"};                // 输出文件路径（交易记录）
    /* 命令行参数部分结束 */

    /* 初始化 Tick 管理器 */
    TickManager *tick_manager;
    // tick_manager = new RandomTickManager(); // 使用随机生成的Tick数据
    tick_manager = new YahooFinanceFileTickManager(input_file); // 从文件读取Tick数据

    /* 初始化交易策略 */
    Strategy *strategy;
    // strategy = new RandomBuySellStrategy(); // 随机买卖策略
    strategy = new SmaStrategy(); // 简单移动平均（SMA）交易策略

```

```

/* 初始化虚拟银行账户 */
VirtualBank *virtual_bank;
REAL bank_balance = 100000; // 初始银行余额为10万
virtual_bank = new VirtualBank(bank_balance);

/* 初始化交易管理器 */
TradeManager *trade_manager;
trade_manager = new PaperTradeManager(virtual_bank); // 纸面交易管理器，模拟交易

Tick last_traded_tick; // 保存上一次交易的Tick数据
/* 核心事件循环 */
while (tick_manager->hasNextTick()) {
    last_traded_tick = tick_manager->getNextTick(); // 获取下一个Tick数据
    // t.dump(); // 可选：输出Tick信息，用于调试

    Trade tr = strategy->processTick(last_traded_tick); // 使用策略处理当前Tick
    trade_manager->performTrade(tr); // 执行交易
}

// 结算交易，平仓
trade_manager->squareOff(last_traded_tick);
// 输出所有交易记录
trade_manager->dumpTrades();

// 计算并输出最终的银行余额和盈亏情况
string final_bank_balance = to_string(virtual_bank->getBankBalance());
string profit_or_loss =
    to_string(virtual_bank->getBankBalance() - bank_balance);

cout << "Total Bank Balance: " << final_bank_balance << endl; // 输出最终余额
cout << "Profit or loss: " << profit_or_loss << endl; // 输出总盈亏

return 0;
}

```

## In "RandomBuySellStrategy.cpp"

### 原代码分析

```

#include "../include/RandomBuySellStrategy.h"
#include "Trade.h"
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <random>

using namespace std;
using Move = Trade::Move; // 使用别名，简化代码

// 随机买卖策略的核心方法：根据随机数生成买入、卖出或观望信号
Trade RandomBuySellStrategy::processTick(Tick tick) {

    // 随机数生成器的初始化（可选注释的代码保留）

```

```

// static random_device rd; // 随机设备
// static mt19937 mt(rd()); // 随机数引擎

// 固定随机数种子，确保结果可复现
static seed_seq seed{43}; // 使用固定种子43
static mt19937 mt(seed); // 使用 Mersenne Twister 随机数生成器

int randomMove;           // 随机生成的动作类型
int quantity = 10;        // 每次交易的默认数量
Trade::Move mv;           // 保存生成的交易动作（买入、卖出或观望）

// 旧的随机数生成方法（基于 time(NULL)，已注释）
// srand((unsigned int)time(NULL));
// randomMove = min + (rand() % static_cast<int>(max - min + 1));

// 生成随机数 1 到 3 之间，用于决定交易动作
uniform_int_distribution<int> dist(1, 3);
randomMove = dist(mt);

// 根据随机数生成对应的信号
if (randomMove == 1) {
    mv = Move::BUY; // 生成买入信号
    if (DEBUG_FLAG) {
        cout << "RandomBuySellStrategy: BUY SIGNAL generated" << endl;
    }
}

if (randomMove == 2) {
    mv = Move::SELL; // 生成卖出信号
    if (DEBUG_FLAG) {
        cout << "RandomBuySellStrategy: SELL SIGNAL generated" << endl;
    }
}

if (randomMove == 3) {
    mv = Move::PASS; // 生成观望信号
    if (DEBUG_FLAG) {
        cout << "RandomBuySellStrategy: PASS SIGNAL generated" << endl;
    }
}

// 返回生成的交易对象，包括动作类型、数量和当前 Tick 数据
return Trade(mv, quantity, tick);
}

```

## In "RandomTickManager.cpp"

### 原代码注释

```

#include "../include/RandomTickManager.h"
#include <cstdlib>
#include <iostream>
#include <random>

```

```

// 生成一个随机整数
int RandomTickManager::genRandomInt() {
    // 使用现代 C++ 随机数生成器代替旧的 rand() 方法
    // srand(time(NULL)); // 旧的随机数种子初始化方法，已被注释掉
    return distribution_int(generator); // 通过预定义的分布和生成器返回随机整数
}

// 生成一个随机浮点数
float RandomTickManager::genRandomFloat() {
    // float r = static_cast<float>(rand()); // 旧的 rand() 方法，已被注释掉
    return distribution_double(generator); // 通过预定义的分布和生成器返回随机浮点数
}

// 生成并返回一个随机 Tick 数据
Tick RandomTickManager::getNextTick() {
    Tick random_tick; // 创建一个 Tick 对象，用于保存生成的随机数据

    // 随机生成 Tick 数据的各个字段
    random_tick.open = genRandomFloat();           // 随机生成开盘价
    random_tick.close = genRandomFloat();           // 随机生成收盘价
    random_tick.high = genRandomFloat();            // 随机生成最高价
    random_tick.adjust_close = genRandomFloat();    // 随机生成调整后的收盘价
    random_tick.volume = genRandomInt();            // 随机生成交易量
    random_tick.time = genRandomInt();              // 随机生成时间戳

    return random_tick; // 返回生成的随机 Tick 数据
}

```

## In "SmaStrategy.cpp"

### 原代码注释

```

#include "../include/SmaStrategy.h"
#include "../include/Trade.h"
#include <iostream>

using namespace std;

using Move = Trade::Move;

// SmaStrategy 类的 processTick 方法：处理每个 Tick 数据
Trade SmaStrategy::processTick(Tick tick) {
    int quantity = 10; // 每次交易的数量
    int smaDayRange = 5; // 移动平均线的天数范围
    Trade::Move mv; // 交易动作（买入、卖出或跳过）

    // 更新累计 Tick 计数和总收盘价
    this->tickCount += 1;
    this->totalClosingPrice += tick.close;

    // 以下代码块：若当前价格相比买入价有较大利润，则安全退出（卖出）
    if (this->lastBoughtPrice != 0 && (tick.close - this->lastBoughtPrice) > 10) {
        mv = Move::SELL; // 卖出信号
        if (DEBUG_FLAG) {

```

```

        cout << "SmaStrategy: Safe SELL SIGNAL generated" << endl; // 调试输出
    }
    return Trade(mv, quantity, tick); // 返回卖出交易
}

// 当收集到足够的天数（smaDayRange）后计算移动平均
if (this->tickCount >= smaDayRange) {
    REAL ma = this->totalClosingPrice / smaDayRange; // 计算移动平均
    this->tickCount = 0; // 重置 Tick 计数
    this->totalClosingPrice = 0; // 重置总收盘价

    // 如果当前价格高于移动平均线，生成买入信号
    if (tick.close > ma) {
        this->isPosition = true; // 标记当前持仓状态
        mv = Move::BUY; // 买入信号
        this->lastBoughtPrice = tick.close; // 记录买入价格
        if (DEBUG_FLAG) {
            cout << "SmaStrategy: BUY SIGNAL generated" << endl; // 调试输出
        }
    } else {
        mv = Move::PASS; // 当前价格低于移动平均线，跳过不交易
        if (DEBUG_FLAG) {
            cout << "SmaStrategy: PASS SIGNAL generated" << endl; // 调试输出
        }
    }
} else {
    mv = Move::PASS; // 未达到 smaDayRange 天数，跳过不交易
    if (DEBUG_FLAG) {
        cout << "SmaStrategy: PASS SIGNAL generated" << endl; // 调试输出
    }
}

// 返回当前 Tick 对应的交易动作
return Trade(mv, quantity, tick);
}

```

## In "Tick.cpp"

### 原代码注释

```

#include "../include/Tick.h"
#include <iostream>

using namespace std;

// Tick 类的 dump 方法：输出 Tick 数据的所有字段值
void Tick::dump() {
    cout << "Dumping tick values" << endl; // 输出提示信息：开始打印 Tick 数据

    // 输出时间戳
    cout << "Timestamp - " << this->time << endl;

    // 输出开盘价
    cout << "Open - " << to_string(this->open) << endl;
}

```

```

// 输出最高价
cout << "High - " << to_string(this->high) << endl;

// 输出最低价
cout << "Low - " << to_string(this->low) << endl;

// 输出收盘价
cout << "Close - " << to_string(this->close) << endl;

// 输出调整后的收盘价
cout << "Adjust Close - " << to_string(this->adjust_close) << endl;

// 输出成交量
cout << "Volume - " << this->volume << endl;

// 输出一个空行，增加可读性
cout << endl;
}

```

## In "TradeManager.cpp"

### 原代码注释

```

#include "../include/TradeManager.h"
#include "../include/Trade.h"
#include <iostream>

using namespace std;

using Move = Trade::Move;

// 执行交易的方法，根据交易类型（BUY或SELL）调用内部方法
int TradeManager::performTrade(Trade trade) {
    if (trade.mv == Move::BUY) { // 如果交易类型为“买入”
        this->performBuyInternal(trade); // 调用买入交易的内部方法
    }
    if (trade.mv == Move::SELL) { // 如果交易类型为“卖出”
        this->performSellInternal(trade); // 调用卖出交易的内部方法
    }

    return 0; // 返回 0 表示交易完成
}

// 执行买入交易的内部方法
int TradeManager::performBuyInternal(Trade trade) {
    if (this->canBuy(trade)) { // 检查是否满足买入条件
        if (DEBUG_FLAG) { // 如果开启调试模式，打印买入日志
            string trade_log = "Bought - " + to_string(trade.quantity) + " at " +
                                to_string(trade.tick.close) + " on " +
                                to_string(trade.tick.time);
        }

        this->trades.push_back(trade); // 将交易记录添加到交易列表中
    }
}

```

```

        this->quantity += trade.quantity; // 更新持有数量
        this->performBuy(trade); // 执行实际的买入操作
        return 1; // 返回 1 表示买入成功
    }

    return 0; // 返回 0 表示买入失败
}

// 执行卖出交易的内部方法
int TradeManager::performSellInternal(Trade trade) {
    if (this->canSell(trade)) { // 检查是否满足卖出条件
        if (DEBUG_FLAG) { // 如果开启调试模式, 打印卖出日志
            string trade_log = "Sold - " + to_string(trade.quantity) + " at " +
                                to_string(trade.tick.close) + " on " +
                                to_string(trade.tick.time);
        }

        this->trades.push_back(trade); // 将交易记录添加到交易列表中
        this->quantity -= trade.quantity; // 更新持有数量
        this->performSell(trade); // 执行实际的卖出操作
        return 1; // 返回 1 表示卖出成功
    }

    return 0; // 返回 0 表示卖出失败
}

// 输出所有交易记录的方法
void TradeManager::dumpTrades() {
    cout << "Printing all the trades" << endl; // 打印提示信息
    cout << endl;

    // 遍历所有交易记录并输出
    for (auto trade : this->trades) {
        trade.dump(); // 调用 Trade 类的 dump 方法输出交易信息
        cout << endl; // 每条交易输出后换行
    }
    cout << endl; // 最后换行, 增加可读性
}

```

## In "VirtualBank.cpp"

### 原代码注释

```

#include "../include/VirtualBank.h"
#include <iostream>

// 构造函数: 初始化银行的初始余额
VirtualBank::VirtualBank(REAL startingBalance) {
    this->totalBalance = startingBalance; // 设置初始余额
}

// 银行存款操作
bool VirtualBank::bankCredit(REAL amount) {
    this->totalBalance += amount; // 将存入的金额添加到当前余额
}

```

```

        return true; // 返回 true 表示存款成功
    }

    // 检查是否可以扣款
    bool VirtualBank::bankCanDebit(REAL amount) {
        if (this->totalBalance >= amount) { // 如果当前余额大于等于要扣除的金额
            return true; // 返回 true 表示可以扣款
        }

        return false; // 返回 false 表示余额不足，无法扣款
    }

    // 银行扣款操作
    bool VirtualBank::bankDebit(REAL amount) {
        if (this->totalBalance >= amount) { // 检查当前余额是否足够扣款
            this->totalBalance -= amount; // 从余额中扣除指定金额
            return true; // 返回 true 表示扣款成功
        }

        return false; // 返回 false 表示扣款失败（余额不足）
    }

    // 获取当前银行余额
    REAL VirtualBank::getBankBalance() {
        return this->totalBalance; // 返回当前余额
    }
}

```

## In "YahooFinanceFileTickManager.cpp"

### 原代码分析

```

#include <algorithm>
#include <fstream>
#include <iostream>
#include <sstream>
#include <vector>

// 注：移除方法: Tick getNextTick(Tick last_tick);

using namespace std;

#include "../include/CurrencyConversions.h"
#include "../include/YahooFinanceFileTickManager.h"

// 构造函数：从指定的文件路径加载数据
YahooFinanceFileTickManager::YahooFinanceFileTickManager(string file_path) {
    if (DEBUG_FLAG) {
        cout << "使用 YahooFinanceFileTickManager 作为默认的 TickManager" << endl;
    }
    ifstream input_file(file_path); // 打开输入文件
    int first_line = 0;
    for (string line; getline(input_file, line);) {
        // 跳过第一行，因为它包含表头
        if (first_line == 0) {

```

```

        first_line++;
        continue;
    }
    // Yahoo 财务数据有时会包含 "null" 数据, 忽略这些行
    if (line.find("null") != string::npos) {
        if (DEBUG_FLAG) {
            cout << "发现空的K线数据, 忽略该行 - " << line << endl;
        }
        continue;
    }

    if (DEBUG_FLAG) {
        cout << "尝试解析行数据 - " << line << endl;
    }

    // 将解析出的数据存储到 tick_store 中
    this->tick_store.push_back(parseTickFromString(line));
}

// 将日期字符串解析为整数
INT YahooFinanceFileTickManager::parseDateFromString(string line) {
    line.erase(remove(line.begin(), line.end(), '-'), line.end()); // 删除日期中的 "-"
    return stol(line); // 转换为整数
}

// 从CSV行字符串中解析出 Tick 对象
Tick YahooFinanceFileTickManager::parseTickFromString(string line) {
    Tick parsed_tick; // 创建一个空的 Tick 对象
    vector<string> tokens; // 用于存储分割后的CSV数据
    stringstream string_stream(line); // 将CSV行数据转为字符串流
    string token;
    int col = 0; // 用于检查CSV列索引

    // 按逗号分割CSV行数据
    while (getline(string_stream, token, ',')) {

        if (col == 0) {
            // 第一列是日期
            parsed_tick.time = parseDateFromString(token);
        } else if (col == 1) {
            // 第二列是开盘价
            CurrencyConversions::removeCommasFromCurrency(token); // 去除货币符号中的逗号
            parsed_tick.open = stold(token); // 将字符串转为长双精度浮点数
        } else if (col == 2) {
            // 第三列是日内最高价
            CurrencyConversions::removeCommasFromCurrency(token);
            parsed_tick.high = stold(token);
        } else if (col == 3) {
            // 第四列是日内最低价
            CurrencyConversions::removeCommasFromCurrency(token);
            parsed_tick.low = stold(token);
        } else if (col == 4) {
            // 第五列是收盘价
            CurrencyConversions::removeCommasFromCurrency(token);
            parsed_tick.close = stold(token);
        }
        col++;
    }
}

```

```

    } else if (col == 5) {
        // 第六列是调整后的收盘价
        CurrencyConversions::removeCommasFromCurrency(token);
        parsed_tick.adjust_close = stold(token);
    } else {
        // 第七列是交易量
        CurrencyConversions::removeCommasFromCurrency(token);
        parsed_tick.volume = stol(token); // 转换为长整型
    }

    col++; // 列索引递增
}

return parsed_tick; // 返回解析后的 Tick 对象
}

// 获取下一条 Tick 数据
Tick YahooFinanceFileTickManager::getNextTick() {
    int idx = this->tick_counter; // 获取当前 tick 索引
    this->tick_counter++; // 索引递增
    return this->tick_store[idx]; // 返回对应的 Tick 数据
}

// 检查是否还有下一条 Tick 数据
bool YahooFinanceFileTickManager::hasNextTick() {
    if (this->tick_counter < this->tick_store.size())
        return true; // 如果还有数据, 返回 true

    return false; // 否则返回 false
}

```

## 1. File Opening (in the constructor)

- **Location:** `ifstream input_file(file_path);`
- **Error Message:** "Unable to open file" or "No such file or directory"
- **Cause:** This error occurs if the file at the specified `file_path` does not exist, cannot be accessed, or there are permission issues with reading the file.
- Exception Handling

:

```

cppCopy codeif (!input_file) {
    cerr << "Error: Unable to open file " << file_path << endl;
    throw runtime_error("File opening failed.");
}

```

## 2. String to Long Conversion (in `parseDateFromString` method)

- **Location:** `return stol(line);`
- **Error Message:** "invalid argument" or "out\_of\_range"
- **Cause:** The `stol()` function could throw an exception if the input string is not a valid number or if the number exceeds the range of `long`. For example, if the string contains non-numeric characters or is too large to be converted to a `long`.
- Exception Handling

:

```
cppCopy codetry {  
    return stol(line);  
} catch (const invalid_argument& e) {  
    cerr << "Error: Invalid date string " << line << endl;  
    throw runtime_error("Date string conversion failed.");  
} catch (const out_of_range& e) {  
    cerr << "Error: Date out of range for " << line << endl;  
    throw runtime_error("Date out of range.");  
}
```

## 3. String to Long Double Conversion (in `parseTickFromString` method)

- **Location:** `parsed_tick.open = stold(token);` (similarly for other price fields)
- **Error Message:** "invalid argument" or "out\_of\_range"
- **Cause:** The `stold()` function could throw an exception if the string is not a valid number, or if the number is too large to be converted to a `long double`. This could happen if the input string contains unexpected characters or is not a properly formatted number.
- Exception Handling

:

```
cppCopy codetry {  
    parsed_tick.open = stold(token);  
} catch (const invalid_argument& e) {  
    cerr << "Error: Invalid number format for " << token << endl;  
    throw runtime_error("Currency value conversion failed.");  
} catch (const out_of_range& e) {  
    cerr << "Error: Currency value out of range for " << token << endl;  
    throw runtime_error("Currency value out of range.");  
}
```

## 4. Out of Range Access (in `getNextTick` method)

- **Location:** `return this->tick_store[idx];`
- **Error Message:** `"out_of_range"`
- **Cause:** This error could occur if `tick_counter` exceeds the size of the `tick_store` vector. If there are no more ticks available and the method is called, it would access an invalid index.
- Exception Handling

:

```
cppCopy codeif (idx >= this->tick_store.size()) {  
    cerr << "Error: Index out of range while accessing ticks." << endl;  
    throw out_of_range("No more ticks available.");  
}  
return this->tick_store[idx];
```

## 5. Invalid Data in CSV Line (in `parseTickFromString` method)

- Location  
: Inside the loop where columns are processed:

```
cppCopy codeCurrencyConversions::removeCommasFromCurrency(token);  
parsed_tick.open = stold(token);
```

- **Error Message:** `"invalid_argument"` or `"bad_cast"`
- **Cause:** The method assumes that all tokens for prices are valid numerical values. If the data is malformed, such as containing invalid characters (like letters) or missing values, it could cause a failure during conversion.
- Exception Handling

:

```
cppCopy codetry {  
    CurrencyConversions::removeCommasFromCurrency(token);  
    parsed_tick.open = stold(token);  
} catch (const invalid_argument& e) {  
    cerr << "Error: Invalid currency value in line: " << line << endl;  
    throw runtime_error("Currency value conversion failed.");  
} catch (const out_of_range& e) {  
    cerr << "Error: Currency value out of range in line: " << line << endl;  
    throw runtime_error("Currency value out of range.");  
}
```

## Writing to `exceptions.txt`:

The following is a sample content that you would write into `exceptions.txt`.

```
vbnetCopy code1. File Opening Error:  
- Error Message: Unable to open file
```

- Cause: File not found or permission issues
- Exception Handling:
 

```
ifstream input_file(file_path);
if (!input_file) {
    cerr << "Error: Unable to open file " << file_path << endl;
    throw runtime_error("File opening failed.");
}
```

## 2. Date Conversion Error:

- Error Message: invalid argument or out\_of\_range
- Cause: Invalid date string or out of range value
- Exception Handling:
 

```
try {
    return stol(line);
} catch (const invalid_argument& e) {
    cerr << "Error: Invalid date string " << line << endl;
    throw runtime_error("Date string conversion failed.");
} catch (const out_of_range& e) {
    cerr << "Error: Date out of range for " << line << endl;
    throw runtime_error("Date out of range.");
}
```

## 3. Currency Value Conversion Error:

- Error Message: invalid argument or out\_of\_range
- Cause: Invalid or malformed currency value in CSV data
- Exception Handling:
 

```
try {
    parsed_tick.open = stold(token);
} catch (const invalid_argument& e) {
    cerr << "Error: Invalid number format for " << token << endl;
    throw runtime_error("Currency value conversion failed.");
} catch (const out_of_range& e) {
    cerr << "Error: Currency value out of range for " << token << endl;
    throw runtime_error("Currency value out of range.");
}
```

## 4. Index Out of Range:

- Error Message: out\_of\_range
- Cause: Attempt to access an invalid index in tick\_store
- Exception Handling:
 

```
if (idx >= this->tick_store.size()) {
    cerr << "Error: Index out of range while accessing ticks." << endl;
    throw out_of_range("No more ticks available.");
}
```

## 5. Invalid Data in CSV:

- Error Message: invalid argument or bad\_cast
- Cause: Malformed or missing values in CSV line
- Exception Handling:
 

```
try {
    parsed_tick.open = stold(token);
} catch (const invalid_argument& e) {
    cerr << "Error: Invalid currency value in line: " << line << endl;
    throw runtime_error("Currency value conversion failed.");
} catch (const out_of_range& e) {
    cerr << "Error: Currency value out of range in line: " << line << endl;
```

```
        throw runtime_error("Currency value out of range.");
    }
```

## In "CurrencyConversions.cpp"

以下是针对提供的 `CurrencyConversions` 类中可能发生异常的五个地方的详细分析，并包括每个异常可能的错误消息、异常原因及处理方法。

### 1. 字符串处理时的 `erase` 错误（在 `removeCommasFromCurrency` 方法中）

- **位置:** `str.erase(remove(str.begin(), str.end(), ','), str.end());`
- **错误消息:** `"out_of_range"` 或 `"invalid_argument"`
- **原因:** `erase` 和 `remove` 方法会修改传入的字符串。如果传入的字符串为空，或者存在不合法的字符，也可能导致 `erase` 操作失败。尽管这种错误较少发生，但它仍然是潜在的风险。
- 异常处理

:

```
cppCopy codetry {
    str.erase(remove(str.begin(), str.end(), ','), str.end());
} catch (const std::out_of_range& e) {
    cerr << "错误: 字符串处理超出范围" << endl;
    throw runtime_error("字符串处理失败");
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的参数传入" << endl;
    throw runtime_error("无效的参数");
}
```

### 2. 字符串流转换失败（在 `readwithPrecision` 方法中）

- **位置:** `sstream >> d;`
- **错误消息:** `"invalid argument"` 或 `"bad cast"`
- **原因:** 在将字符串流中的内容转换为 `REAL` 类型时，如果字符串内容无法转换成有效的数字（例如字符串包含非数字字符或格式不正确），`stringstream` 会发生转换失败。
- 异常处理

:

```
cppCopy codetry {
    sstream >> d;
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无法从字符串转换为数字" << endl;
    throw runtime_error("转换失败");
} catch (const std::bad_cast& e) {
    cerr << "错误: 类型转换失败" << endl;
    throw runtime_error("类型转换失败");
}
```

### 3. 无效的精度值（在 `readWithPrecision` 方法中）

- **位置:** `sstream << setprecision(p) << fixed << str << endl;`
- **错误消息:** `"invalid argument"`
- **原因:** 如果精度 `p` 传入了一个无效的值，或者 `setprecision` 参数超出了合理范围，可能会导致 `stringstream` 出现无法预期的行为。
- 异常处理

:

```
cppCopy codetry {
    sstream << setprecision(p) << fixed << str << endl;
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的精度值 " << p << endl;
    throw runtime_error("无效的精度值");
}
```

### 4. 内存分配失败（在字符串流操作中）

- **位置:** `stringstream sstream;`
- **错误消息:** `"bad_alloc"`
- **原因:** `stringstream` 依赖于内存分配来处理流的操作。如果系统内存不足，可能会抛出 `bad_alloc` 异常。
- 异常处理

:

```
cppCopy codetry {
    stringstream sstream;
    sstream << setprecision(p) << fixed << str << endl;
} catch (const std::bad_alloc& e) {
    cerr << "错误: 内存分配失败" << endl;
    throw runtime_error("内存分配失败");
}
```

### 5. 空字符串输入（在 `readWithPrecision` 方法中）

- **位置:** `sstream >> d;`
- **错误消息:** `"invalid argument"`
- **原因:** 如果传入的字符串为空，或者仅包含空格字符，`stringstream` 会尝试读取一个无效的值，导致转换失败。
- 异常处理

:

```
cppCopy codeif (str.empty()) {
    cerr << "错误: 输入字符串为空" << endl;
    throw runtime_error("输入字符串为空");
}
```

## 写入 exceptions.txt 文件:

以下是 exceptions.txt 文件的内容, 包含所有可能的异常信息、原因和处理方法。

phpCopy code1. 字符串处理错误:

- 错误消息: out\_of\_range 或 invalid\_argument
- 原因: 对空字符串或无效参数进行操作
- 异常处理:

```
try {
    str.erase(remove(str.begin(), str.end(), ','), str.end());
} catch (const std::out_of_range& e) {
    cerr << "错误: 字符串处理超出范围" << endl;
    throw runtime_error("字符串处理失败");
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的参数传入" << endl;
    throw runtime_error("无效的参数");
}
```

2. 字符串流转换失败:

- 错误消息: invalid\_argument 或 bad\_cast
- 原因: 字符串无法转换为数字
- 异常处理:

```
try {
    sstream >> d;
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无法从字符串转换为数字" << endl;
    throw runtime_error("转换失败");
} catch (const std::bad_cast& e) {
    cerr << "错误: 类型转换失败" << endl;
    throw runtime_error("类型转换失败");
}
```

3. 无效的精度值:

- 错误消息: invalid\_argument
- 原因: 传入了无效的精度值
- 异常处理:

```
try {
    sstream << setprecision(p) << fixed << str << endl;
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的精度值 " << p << endl;
    throw runtime_error("无效的精度值");
}
```

4. 内存分配失败:

- 错误消息: bad\_alloc
- 原因: 内存不足导致字符串流分配失败
- 异常处理:

```
try {
    stringstream sstream;
    sstream << setprecision(p) << fixed << str << endl;
} catch (const std::bad_alloc& e) {
    cerr << "错误: 内存分配失败" << endl;
    throw runtime_error("内存分配失败");
}
```

#### 5. 空字符串输入:

- 错误消息: `invalid argument`
- 原因: 输入的字符串为空
- 异常处理:

```
if (str.empty()) {  
    cerr << "错误: 输入字符串为空" << endl;  
    throw runtime_error("输入字符串为空");  
}
```

## 原代码分析

```
#include "../include/CurrencyConversions.h"  
#include <algorithm>  
#include <iomanip>  
#include <iostream>  
#include <sstream>  
  
using namespace std;  
  
// 移除字符串中的逗号, 用于处理货币格式  
void CurrencyConversions::removeCommasFromCurrency(string &str) {  
    // 使用 `remove` 算法移除逗号并调整字符串末尾  
    str.erase(remove(str.begin(), str.end(), ','), str.end());  
}  
  
// 读取带有指定精度的字符串, 并将其转换为 REAL 类型  
REAL CurrencyConversions::readWithPrecision(string const &str, size_t const p) {  
    stringstream sstream; // 创建字符串流  
  
    // 设置精度, 并将字符串写入字符串流  
    sstream << setprecision(p) << fixed << str << endl;  
  
    REAL d; // 用于存储转换后的数值  
  
    // 从字符串流中读取数值, 转换为 REAL 类型  
    sstream >> d;  
  
    return d; // 返回转换后的数值  
}
```

## In "PaperTradeManager.cpp"

## 原代码分析

```
#include "../include/PaperTradeManager.h"  
#include "../include/Trade.h"  
#include <iostream>  
  
using namespace std;  
  
// 构造函数: 初始化虚拟银行和持仓数量  
PaperTradeManager::PaperTradeManager(VirtualBank *v_bank) {  
    this->virtual_bank = v_bank; // 关联虚拟银行账户
```

```

        this->holdQuantity = 0;          // 初始持仓数量为0
    }

    // 执行买入操作
    void PaperTradeManager::performBuy(Trade trade) {
        cout << "Paper trade buy" << endl; // 输出买入操作信息
        // 从虚拟银行账户中扣除购买股票所需的资金
        this->virtual_bank->bankDebit(trade.quantity * trade.tick.close);
        // 增加持仓数量
        this->holdQuantity += trade.quantity;
    }

    // 执行卖出操作
    void PaperTradeManager::performSell(Trade trade) {
        cout << "Paper trade sell" << endl; // 输出卖出操作信息
        // 向虚拟银行账户中存入卖出股票所获得的资金
        this->virtual_bank->bankCredit(trade.quantity * trade.tick.close);
        // 减少持仓数量
        this->holdQuantity -= trade.quantity;
    }

    // 判断是否可以执行买入操作
    bool PaperTradeManager::canBuy(Trade trade) {
        // 检查虚拟银行账户中是否有足够的资金进行买入
        if (this->virtual_bank->bankCanDebit(trade.quantity * trade.tick.close)) {
            return true; // 资金充足，可以买入
        }

        // 如果 DEBUG_FLAG 为真，输出错误信息
        if (DEBUG_FLAG) {
            cout << "Could not perform a BUY signal, probably insufficient funds!"
                 << endl; // 资金不足，无法买入
        }

        return false; // 资金不足，不能买入
    }

    // 判断是否可以执行卖出操作
    bool PaperTradeManager::canSell(Trade trade) {
        // 检查当前持仓数量是否足够执行卖出操作
        if (trade.quantity <= this->holdQuantity) {
            return true; // 持仓充足，可以卖出
        }

        // 如果 DEBUG_FLAG 为真，输出错误信息
        if (DEBUG_FLAG) {
            cout << "Could not perform a SELL signal, you cant sell what you dont have"
                 << endl; // 持仓不足，无法卖出
        }

        return false; // 持仓不足，不能卖出
    }

    // 平仓操作：卖出当前所有持仓
    bool PaperTradeManager::squareOff(Tick last_tick) {
        using Move = Trade::Move;

```

```
// 如果当前持仓数量大于0，创建卖出交易进行平仓
if (this->holdQuantity > 0) {
    Trade squareoff_trade(Move::SELL, this->holdQuantity, last_tick);
    this->performTrade(squareoff_trade); // 执行卖出操作
}
return false; // 平仓完成
}
```

## 1. 虚拟银行账户无法扣款（在 `performBuy` 方法中）

- **位置:** `this->virtual_bank->bankDebit(trade.quantity * trade.tick.close);`
- **错误消息:** "insufficient funds" 或 "bank error"
- **原因:** 如果虚拟银行账户余额不足以进行购买，或者 `bankDebit` 函数内部发生了错误（例如数据库连接问题、负数金额等），可能会抛出异常。
- **异常处理:**

```
cppCopy codetry {
    this->virtual_bank->bankDebit(trade.quantity * trade.tick.close);
} catch (const std::runtime_error& e) {
    cerr << "错误：虚拟银行扣款失败 - " << e.what() << endl;
    throw runtime_error("虚拟银行扣款失败");
}
```

## 2. 虚拟银行账户无法存款（在 `performSell` 方法中）

- **位置:** `this->virtual_bank->bankCredit(trade.quantity * trade.tick.close);`
- **错误消息:** "bank error" 或 "transaction failure"
- **原因:** 如果银行存款失败，例如银行连接问题，或者发生了系统故障，可能会抛出异常。
- **异常处理:**

```
cppCopy codetry {
    this->virtual_bank->bankCredit(trade.quantity * trade.tick.close);
} catch (const std::runtime_error& e) {
    cerr << "错误：虚拟银行存款失败 - " << e.what() << endl;
    throw runtime_error("虚拟银行存款失败");
}
```

## 3. 虚拟银行无法扣款（在 `canBuy` 方法中）

- **位置:** `this->virtual_bank->bankCanDebit(trade.quantity * trade.tick.close)`
- **错误消息:** "insufficient funds" 或 "bank error"
- **原因:** 如果 `bankCanDebit` 方法检查账户余额时遇到异常（例如连接失败或计算错误），可能会导致异常抛出。
- **异常处理:**

```
cppCopy code
try {
    if (this->virtual_bank->bankCanDebit(trade.quantity * trade.tick.close)) {
        return true;
    }
} catch (const std::runtime_error& e) {
    cerr << "错误: 无法检查账户余额 - " << e.what() << endl;
    throw runtime_error("无法检查账户余额");
}
```

## 4. 虚拟银行无法进行交易 (在 `canSell` 方法中)

- **位置:** `if (trade.quantity <= this->holdQuantity)`
- **错误消息:** `"insufficient holdings"` 或 `"invalid quantity"`
- **原因:** 如果交易的数量大于持有的数量, 或者交易数据本身有问题 (如负值), 可能会导致异常。
- **异常处理:**

```
cppCopy code
try {
    if (trade.quantity <= this->holdQuantity) {
        return true;
    }
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的交易数量 - " << e.what() << endl;
    throw runtime_error("无效的交易数量");
}
```

## 5. 在 `squareOff` 方法中执行交易失败

- **位置:** `this->performTrade(squareoff_trade);`
- **错误消息:** `"trade execution failed"` 或 `"transaction error"`
- **原因:** `performTrade` 方法可能因银行操作失败、虚拟账户余额问题或其他原因而无法成功执行交易。
- **异常处理:**

```
cppCopy code
try {
    this->performTrade(squareoff_trade);
} catch (const std::runtime_error& e) {
    cerr << "错误: 交易执行失败 - " << e.what() << endl;
    throw runtime_error("交易执行失败");
}
```

## 写入 `exceptions.txt` 文件:

以下是 `exceptions.txt` 文件的内容, 记录了所有可能发生的异常、错误消息、原因和如何处理这些异常的详细内容。

- ```
kotlinCopy code
1. 虚拟银行账户无法扣款:
- 错误消息: insufficient funds 或 bank error
- 原因: 账户余额不足, 或者银行操作失败
- 异常处理:
```

```

try {
    this->virtual_bank->bankDebit(trade.quantity * trade.tick.close);
} catch (const std::runtime_error& e) {
    cerr << "错误: 虚拟银行扣款失败 - " << e.what() << endl;
    throw runtime_error("虚拟银行扣款失败");
}

```

## 2. 虚拟银行账户无法存款:

- 错误消息: `bank error` 或 `transaction failure`
- 原因: 存款失败或银行操作问题
- 异常处理:

```

try {
    this->virtual_bank->bankCredit(trade.quantity * trade.tick.close);
} catch (const std::runtime_error& e) {
    cerr << "错误: 虚拟银行存款失败 - " << e.what() << endl;
    throw runtime_error("虚拟银行存款失败");
}

```

## 3. 虚拟银行无法扣款:

- 错误消息: `insufficient funds` 或 `bank error`
- 原因: 无法检查账户余额, 可能是系统错误或连接问题
- 异常处理:

```

try {
    if (this->virtual_bank->bankCanDebit(trade.quantity * trade.tick.close)) {
        return true;
    }
} catch (const std::runtime_error& e) {
    cerr << "错误: 无法检查账户余额 - " << e.what() << endl;
    throw runtime_error("无法检查账户余额");
}

```

## 4. 虚拟银行无法进行交易:

- 错误消息: `insufficient holdings` 或 `invalid quantity`
- 原因: 交易数量大于持有数量, 或者交易数据无效
- 异常处理:

```

try {
    if (trade.quantity <= this->holdQuantity) {
        return true;
    }
} catch (const std::invalid_argument& e) {
    cerr << "错误: 无效的交易数量 - " << e.what() << endl;
    throw runtime_error("无效的交易数量");
}

```

## 5. 在 `squareOff` 方法中执行交易失败:

- 错误消息: `trade execution failed` 或 `transaction error`
- 原因: 银行操作失败、账户余额问题或其他交易执行错误
- 异常处理:

```

try {
    this->performTrade(squareoff_trade);
} catch (const std::runtime_error& e) {
    cerr << "错误: 交易执行失败 - " << e.what() << endl;
    throw runtime_error("交易执行失败");
}

```

